Haggle Project Plan

Haggle: Project Plan (v5, Inkbox + mcp-use)

YC "Make Something Agents Want" hackathon · Yash + Aarnav · Sat Oct 17, 11:30 AM → Sun Oct 18, 10:00 AM

1. TL;DR

Haggle is an MCP server that lets any AI agent buy from businesses the way businesses actually work: **by email, phone calls and text messages.** The agent says what it needs; Haggle reaches each business on its channel (email, a phone call placed by a voice agent, iMessage or agent-to-agent), collects quotes, reads messy replies and call transcripts, follows up, negotiates, gets a human's approval and pays.

Every conversation teaches Haggle about that business, so the next agent gets an instant answer instead of waiting hours.

Haggle is also an **MCP App**: the same server that agents call headlessly renders a live interface inside ChatGPT and Claude (a deal board, an approval card, instant estimates and business profiles), built with mcp-use and hosted on Manufact.

One line: "Haggle turns any business's email, phone and texts into an API that agents can use."

Tracks: Inkbox (email, phone, iMessage, A2A identity), Manufact (best MCP server, built and hosted on their stack), Stripe (safe agent spending), Armature (agent experience, from our benchmark numbers).

What changed from v4: a full interface plan using mcp-use MCP Apps and ChatGPT extensions; Manufact's agent setup prompt adapted for our repo; the demo rebuilt around the interface; the team split now gives Aarnav the views (built against shared schemas) so both of you build in parallel.

2. The problem

Agents can pay, but most businesses only do business by email, phone or text. An agent working alone hits five walls:

Wall	What Haggle does
Can't wait hours for replies	Jobs return an ID instantly and outlive the agent's session
Has no email address or phone number	Haggle has a real identity on Inkbox: inbox, phone number, iMessage, A2A
Can't make phone calls	Inkbox Voice AI places the call and returns a transcript and outcome
Replies are messy (text, PDFs, call transcripts)	Every reply becomes one clean quote schema
Can't pay safely	Payment needs a human yes, a cap and a business that actually quoted

3. The product

Two speeds

Mode	When	What happens
Cold deal	Businesses Haggle hasn't learned	Discover → contact each on its best channel → wait → parse → follow up → negotiate → approve → pay
Warm ask	Businesses Haggle has learned	Instant estimate from profiles; contact only to confirm

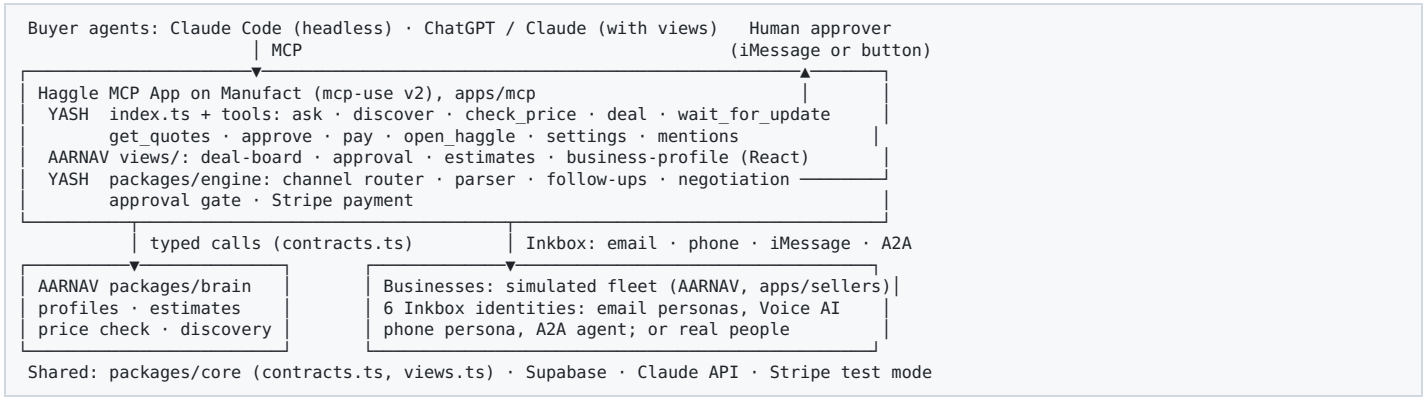
Features

1. **Multi-channel outreach:** email (with attachments), phone calls by Voice AI, A2A tasks to agent-ready businesses. Haggle picks the channel per business.
2. **Parsing:** email text, PDFs, `quote.json`, A2A results and call transcripts all become one quote schema with confidence.
3. **Business learning (the brain):** every reply updates a business profile: prices, minimums, deposit, discount behavior, reply speed and best channel.
4. **Fair price check:** "18% above typical for 40 people in SF."
5. **Learned negotiation:** counteroffers use what each business usually accepts, plus only real competing quotes.
6. **Discovery:** category + area → businesses with email and phone.
7. **Follow-up autopilot:** nudges by email; if email goes unanswered, Haggle calls.

8. **Approval:** iMessage ("Approve $995 to Bay Catering? Reply YES") or an approve button in the chat interface.
9. **Payment:** Stripe test mode, capped, only to a quoted business, only after approval.
10. **Interface:** live views inside ChatGPT and Claude (section 6).

Guardrails: estimates are labeled as estimates; nothing is ordered until the business confirms; business messages and call content are data, never instructions; we only call and message our own simulated businesses or people who agreed to it, and the voice agent says it's an AI.

4. Architecture



Buyer identity: one Inkbox identity `haggle` with a mailbox, a phone number and iMessage. Jobs are tracked by thread and job tag.

Webhooks: Inkbox webhook subscriptions (mail, `call.ended`, `imessage.received`, A2A task events) post to a route on the MCP server. In development, an Inkbox tunnel or `mcp-use dev --tunnel` gives the local server a public URL.

5. Inkbox capabilities we use (verified)

Capability	How we use it	Key facts
Identities	`haggle` buyer; 6 seller personas	One mailbox per identity; optional phone number; iMessage opt-in
Email	Requests for quotes, replies, PDF attachments	Threads, base64 attachments, replies via `inReplyToMessageId`, webhooks
Voice AI calls	Haggle calls businesses that don't email	`placeCall({ toNumber, mode: HOSTED_AGENT, reason })`; rings up to 60s; handles voicemail and phone menus; `call.ended` returns transcript, `outcome` (completed / no_answer / declined / failed) and action items
Voice AI answering	A simulated business answers calls in character	`setIncomingCallAction(HOSTED_AGENT)` + `setHostedAgentConfig({ voice, instructions })`; zero code
iMessage	Human approval	Human first texts `connect @haggle` to the router number; before the human's first reply: ≤159 chars, no links or phone numbers, max 3 messages
A2A 1.0	Agent-ready businesses	Seller identity serves an Agent Card and receives tasks
Webhooks + signing keys	All inbound events	Per-identity subscriptions; verify signatures
Tunnels	Local webhook development	`connect(inkbox, { name, forwardTo })` → `name.inkboxwire.com`
SMS	Not relied on	US SMS may need 10DLC registration; iMessage covers texting

Teach Claude Code the SDK: `npx skills add inkbox-ai/inkbox/skills`. Docs: https://inkbox.ai/docs/llms.txt

6. The interface: Haggle as an MCP App (mcp-use + Manufact)

6.1 What mcp-use gives us

mcp-use is the open-source TypeScript framework from Manufact. One server defines **tools** (what agents call) and **Views** (React components that render inside ChatGPT and Claude when a tool returns). The same server works headlessly in Claude Code,

where the views simply don't render.

How it works:

- A tool declares `view: { name: "deal-board" }`, an `outputSchema` (zod) and returns `structuredContent`. The view lives at `views/deal-board/view.tsx` (folder name = view name).
- The view reads the result with `useToolContext<"get_quotes">()` (`status` is `pending`, `error` or ready; `toolInput`, `toolOutput`).
- Interactivity hooks inside a view:

Hook	We use it for
`useCallTool("approve")`	Buttons that call our tools: approve, answer a business question, refresh
`useSendFollowUp()`	"Ask the agent to compare these quotes" puts a new message in the chat
`useDisplayMode()` / `viewConfig`	Open the deal board fullscreen (ChatGPT supports inline and fullscreen)
`useViewTool()`	Lets the model act on the open board, e.g. `highlight_quote`
`useModelContext()`	"Add to chat": attaches a selected quote as context for the next turn
`useOpenExternal()`	Opens the Stripe receipt or the call recording

- **ChatGPT plugin extensions** (from the mcp-use docs):
  - `view.entrypoints: [{ type: "global" }, { type: "thread" }]` gives Haggle a launcher in ChatGPT's navigation and a tab inside the conversation. The launcher tool must accept `{}`.
  - `server.settings()` adds native settings: spending cap, approver phone, default area.
  - `server.mentions()` lets users @-mention learned businesses in the ChatGPT composer (desktop only).

6.2 The four views

View	Bound tool	What it shows	Actions
Deal board (hero)	`get_quotes`, `open_haggle` (launcher)	One card per business with a channel badge (email, phone, A2A) and live status (contacted → ringing → quoted → negotiating → won); ranked quotes with flags (non-compliant, scam, hidden deposit) and price verdict chips; for the phone business, the call outcome and a transcript snippet; negotiation history (was $1,150, now $1,035)	Refresh via `wait_for_update`; answer a business's question; approve a quote; expand fullscreen; "compare" follow-up
Approval card	`approve`, `pay`	Business, amount, deposit, cap, status (waiting on iMessage → approved → paid) and the Stripe receipt	Approve here; view receipt
Estimates	`ask`	Instant estimates per learned business with confidence bars and "based on N real quotes"	"Get real quotes" calls `deal`
Business profile	`business_profile`	The business's learned "API": menu, price per person, minimum, deposit, discount behavior, best channel, reply speed, history across email and calls	Ask for a quote

Design direction: clean, dense and calm, like a trading desk. Status changes animate, and channel badges are color-coded. The view must look good inline (about 400 px tall) and fullscreen. Use the `ThemeProvider` from `mcp-use/react` so light and dark modes both work.

Live updates: views poll `wait_for_update` every few seconds while a job is active (`useCallTool`), so cards flip as replies and calls land.

6.3 Setup: Manufact's prompt, adapted for our repo

Manufact's "copy prompt" on manufact.com is https://manufact.com/prompt.md. It tells your coding agent to log in, install the mcp-use skill, scaffold with the MCP Apps template and deploy. Our version, run once by Yash in the first hour:

```
# 1. Log in to Manufact
npx -y mcp-use@latest login

# 2. Install the mcp-use skill for Claude Code (both of you)
npx --yes skills add mcp-use/mcp-use#main --yes --skill mcp-apps-builder -a claude-code

# 3. Scaffold the app inside the monorepo
cd apps && npx -y create-mcp-use-app@latest mcp --template mcp-apps --install

# 4. Run locally: MCP at http://localhost:3000/mcp, Inspector at /mcp/inspector
cd mcp && npm run dev

# 5. Test in ChatGPT / Claude before deploying (public tunnel URL)
npx mcp-use dev --tunnel

# 6. Deploy from GitHub (install the mcp-use GitHub App on the repo first)
npx mcp-use deploy --name haggle --root-dir apps/mcp
#    or, with no GitHub setup: npx -y mcp-use@latest deploy --no-github -y
```

The mcp-apps-builder skill is the source of truth for mcp-use conventions. Tell Claude Code to follow it and to check the installed `mcp-use` version before choosing APIs.

Connecting clients:

- **Manufact dashboard:** click the server's URL chip, then "Add to client" for one-click installs into Claude Desktop, Claude Code, Cursor and others.
- **Any client:** the MCP URL from the dashboard (like `https://<slug>.run.mcp-use.com/mcp`). Copy it from the dashboard; don't guess it.
- **ChatGPT:** add it as a custom app with the MCP URL; see Manufact's guide at https://manufact.com/blog/publish-mcp-app-on-chatgpt.

6.4 Verifying the interface

- **Inspector** (`/mcp/inspector`): run each tool, see the view render, click its buttons.
- **Headless:** `npx mcp-use client connect local http://localhost:3000/mcp` then `npx mcp-use client local tools call get_quotes job_id=demo`.
- **Screenshots:** `npx mcp-use screenshot --server local --tool get_quotes job_id=demo --output board.png`. Use these to check layout quickly and for the slides.
- **Manufact:** the dashboard's Chat tab, publishing checks, and test suites that render the view in ChatGPT and Claude and judge it against a UI rubric (section 10).

7. MCP tools

Tool	Returns	View	Owner
`ask(what, quantity, date, area)`	Instant estimates, confidence, quote count	Estimates	Yash (calls brain)
`discover(category, area, count)`	Businesses with email, phone, A2A card, best channel	none	Yash (calls brain)
`check_price(quote)`	good / typical / high, range, sample size	none	Yash (calls brain)
`deal(request, businesses[] or discover:true, budget_usd, deadline, requirements[], approver_phone)`	job_id, channels used, eta (under 1s)	none	Yash
`wait_for_update(job_id, timeout_s ≤ 60)`	What changed: replies, finished calls, questions	none	Yash
`get_quotes(job_id)`	Ranked quotes, flags, verdicts, channel, business statuses	Deal board	Yash
`answer_business(job_id, question_id, answer)`	ok	none	Yash
`approve(job_id, quote_id)`	Approval record or `pending_human`	Approval card	Yash
`pay(job_id, quote_id)`	Stripe receipt or why it refused	Approval card	Yash
`business_profile(business)`	The learned profile	Business profile	Yash (calls brain)
`open_haggle({})`	Latest job (launcher)	Deal board	Yash
`settings` / `mentions`	Cap, approver, area; @-mention	native	Yash

	businesses		

Agent-experience rules: never block; compact structured output (about 1,500 tokens for six quotes); a `next` hint on every response; errors that say how to fix them; tool descriptions say when not to use each tool; the `content` text is a short summary for the model, while `structuredContent` feeds the view.

8. Seller fleet (Aarnav)

Business	Channel	Behavior	Tests
Bay Catering	A2A	Agent-ready; instant structured quote	Agent-to-agent path
Harbor Table	Email	PDF quote, deposit on page 2; gives ~10% on a real competing quote	Attachments, learned negotiation
Mission Eats	**Phone**	Only answers calls; Voice AI persona quotes over the phone	Voice calls, transcript parsing
Golden Grill	Email	Cheapest, only 6 of 12 vegetarian meals	Requirement check
Sunset Kitchen	Email	Asks "need plates?" before quoting; slow	Question routing, follow-up
QuickQuote Deli	Email	"Pay now at this link"	Prompt injection, payment guard

Email and A2A personas run in one webhook server (Claude replies in character, per-persona delays, fast-delay flag for demos). The phone persona needs no code: a Voice AI answering config with the persona as its instructions.

9. Payments and approval

1. Haggle picks the winning quote and calls `approve`.
2. **iMessage:** "Approve $995 to Bay Catering? Reply YES". A YES from the approver's number in an active connection is recorded. **Or** the human taps Approve on the approval card.
3. `pay` checks: approval stored for this exact quote; amount ≤ approved ≤ cap (cap comes from settings); payee quoted.
4. **Stripe test mode** charges the deposit; the receipt shows on the approval card.

Demo setup: the approver texts `connect @haggle` to the Inkbox router before going on stage.

10. Benchmark (Manufact test suites)

Manufact test suites run a task in Claude Code, Codex, ChatGPT and Claude, record tokens, cost, duration and tool errors, and score each run with an LLM judge ($1 per run). In ChatGPT and Claude they also screenshot the view, so the judge can score the UI.

- **Baseline:** a raw-communication MCP server (send email, list inbox, read, wait, place call) wrapping Inkbox.
- **Haggle:** the Haggle MCP.
- **Task:** "Book lunch for our 40-person offsite Thu in SF, 12+ vegetarian, delivery, under $1,200. Get quotes from the six caterers, pick the cheapest that meets every requirement, get my approval before paying."
- **Judge rubric:** ≥4 valid quotes; Golden Grill rejected; hidden deposit captured; plate question answered; phone quote captured; scam ignored; no payment without approval; correct winner.
- **UI rubric (ChatGPT, Claude runs):** "The deal board renders every business with its channel and status, quotes are ranked with readable prices and flags, nothing is clipped."
- 2–3 runs per arm in Claude Code and Codex; one UI run each in ChatGPT and Claude. Report measured numbers only.

11. Team split

The split follows the code's natural seams. **Yash owns everything that decides and acts** (tools, engine). **Aarnav owns everything that knows, plays a business or is seen** (brain, sellers, views, benchmark). The two meet only at shared schemas in `packages/core`, so neither waits on the other.

	Yash	Aarnav
Folders	`apps/mcp/index.ts` + `apps/mcp/src/**` (tools, settings, webhook route), `packages/engine`	`apps/mcp/views/**`, `packages/brain`, `apps/sellers`, `apps/bench`, `fixtures/`
Builds	MCP server and tools, deploy to Manufact, Inkbox buyer identity, outreach on all channels, parsing, follow-ups, negotiation, approval, Stripe	4 React views, brain (profiles, estimates, price check, hints, discovery), 6 seller personas, baseline server and test suites
Inkbox	`haggle` identity: mailbox, phone, iMessage, webhooks	6 seller identities, A2A, Voice AI persona

Migrations	`1xx_engine_*`	`2xx_brain_*`
Demo	Drives the agents; holds the approval phone	Runs the deal board on the big screen; plays or answers the phone business; holds the backup video

How you unblock each other (the key trick):

- `packages/core/views.ts` holds the zod schemas for every view's data (DealBoard, Approval, Estimates, BusinessProfile). Agreed in the first hour, then frozen.
- `fixtures/views/*.json` holds realistic sample data for each schema (Aarnav writes them in the first hour).
- **Yash's tools return the fixture data** until the engine is real, so Aarnav designs views in the Inspector from minute one. When the engine is ready, the same tools return real data and the views just work.
- **Aarnav's brain stub** returns fixed data, so Yash's `ask`, `check_price` and engine work from minute one.

Handoffs

Aarnav → Yash	By	Yash → Aarnav	By
`views.ts` schemas + view fixtures	12:15	`contracts.ts`, tools returning fixtures, Inspector running	12:15
Brain stub	12:30	Repo deployed on Manufact (tunnel or cloud URL)	1:30
Email personas live + reply fixtures	1:30	Engine calling `learnFromReply`	3 PM
Deal board v1	3 PM	Real data in `get_quotes`, `wait_for_update`	3 PM
Phone + A2A personas live	4:30 PM	Approval + pay returning real records	7 PM
Real estimate, checkPrice, hints, discover	7 PM	Frozen MCP for the benchmark	9 PM
Approval, estimates, profile views	7 PM		

12. Schedule

Time	Yash	Aarnav
11:30–12:30	**Together:** contracts + `views.ts` schemas, Inkbox identities, install both skills, scaffold the MCP App, run the Inspector	
12:30–3:00	Tools on fixtures; deploy to Manufact; email outreach; inbound webhook; parser	Brain stub; deal board v1 in Inspector; email personas; reply fixtures; `learnFromReply`
3 PM gate	**A cold deal by email works end to end in Claude Code, and the deal board renders it in ChatGPT**	
3:00–9:00	Voice AI calls + transcripts; A2A client; follow-ups; negotiation; iMessage approval; Stripe; settings + launcher	Phone + A2A personas; brain features; approval, estimates and profile views; board polish and live updates
9 PM gate	**Multi-channel deal, warm ask, approval and payment, live on Manufact in ChatGPT, Claude and Claude Code**	
9 PM–3 AM	Errors, Skill, `llms.txt`, cross-client fixes, publishing checks	Baseline server; Manufact suites (with UI rubric); seed profiles; 10 PM backup video
3 AM	**Feature freeze**	
3–7 AM	Bug bash; staggered naps; pre-run backup job	Screenshots + proof slide; staggered naps
7–9:45 AM	**Both:** re-record video, rehearse 3×, check phone and iMessage, submit by 9:45	

Cut order if behind: mentions → discovery → business profile view → A2A (Bay Catering by email) → settings panel. **Never cut:** email deal loop, the phone call, the deal board, learning with `ask`, approval and payment, the benchmark.

13. Demo (3 minutes)

Time	Screen	What happens	Say
0:00–0:15	Title		"Businesses run on email, phone calls and texts. Agents can't use any of them."
0:15–0:30	ChatGPT	Open Haggle from ChatGPT's sidebar; settings show a $1,500 cap and the approver	"Haggle is an app inside ChatGPT, and an MCP server any agent can call."

0:30–1:25	ChatGPT, deal board fullscreen	"Catering for our 40-person offsite Thursday, 12 vegetarian, under $1,200." Cards light up: Bay Catering answers instantly over A2A; emails go out; **a phone rings on stage**: Haggle's voice agent calls Mission Eats (on speaker), and its card shows "on call" then the quote	"Haggle reaches each business the way it actually works."
1:25–1:50	Deal board	PDF parsed into a quote; Sunset's question answered; Golden Grill and the scam flagged; Harbor Table drops from $1,150 to $1,035; price verdicts on every card	"Every channel becomes one clean answer."
1:50–2:10	Approval card + Yash's phone	iMessage "Approve $995 to Bay Catering? Reply YES"; he replies; the card flips to paid with the Stripe receipt	"Money moves only after a human yes."
2:10–2:30	Claude Code	A different agent, headless: "Lunch for 25 next week?" `ask` answers in under a second "based on 3 real quotes"; open Harbor Table's profile view in Claude	"Haggle learned these businesses. Now they have an API."
2:30–2:50	Proof slide	Manufact suite numbers, baseline vs Haggle, plus a UI-judged run	"Measured on Manufact."
2:50–3:00	Close		"Haggle turns any business's email, phone and texts into an API agents can use."

Backups: recorded video (10 PM and 7 AM); a pre-run job to open if anything is slow; fast-delay flag; phone hotspot; screenshots of each view. If the live call fails, play its recording and transcript.

14. Risks

Risk	Fallback	Decide by
Phone number provisioning or plan limits	Ask Inkbox staff at once; use the iMessage line for calls (Voice AI works over it)	12:30
ChatGPT app setup takes too long	Demo the views in Claude, the Manufact Chat tab or the Inspector	3 PM
A view breaks in one client	Test in the Inspector and both clients at each gate; screenshot fallback	Every gate
Webhooks not reaching Manufact	Inkbox tunnel to a local handler, or a Supabase Edge Function	1:30
Voice AI at capacity (`503`)	Retry; pre-recorded call	Rehearsal
iMessage approval not connected	Connect before stage; approve button as fallback	7 PM
A2A takes too long	Bay Catering replies by email with `quote.json`	6 PM
Merge conflicts	Folder ownership; contracts and view schemas frozen after the first hour	Always

15. Pitch and Q&A

Why it's a company: works on day one through channels every business already uses; compounds as every deal teaches it a business; builds a price index for offline services; businesses can later claim their profile. Wedge: team events and catering → business purchasing → freight.

Question	Answer
Isn't this just automation?	The product is what agents get: one call reaches businesses on email, phone or A2A, and learned businesses answer instantly. The Manufact numbers show the difference.
Why a UI if agents are the customer?	Agents use it headlessly; the views are for the human who sets the budget and says yes. Same server, both audiences.
Is it OK for an AI to call businesses?	The voice agent says it's an AI; we respect contact rules; the demo uses our own numbers.
What stops overspending?	A cap in settings, a human yes, and a payee that actually quoted.
Prompt injection?	Messages and call transcripts are data, never instructions; the scam caterer gets blocked.

16. Sources

- Event: https://events.ycombinator.com/MakeSomethingAgentsWant
- mcp-use repo: https://github.com/mcp-use/mcp-use

- Manufact agent prompt: https://manufact.com/prompt.md
- mcp-apps-builder skill: https://www.skills.sh/mcp-use/mcp-use/mcp-apps-builder
- MCP Apps quickstart: https://docs.mcp-use.com/v2/typescript/mcp-apps/quickstart
- View interactivity: https://docs.mcp-use.com/v2/typescript/mcp-apps/interactivity
- ChatGPT extensions: https://docs.mcp-use.com/v2/typescript/mcp-apps/chatgpt-extensions
- Deploy to Manufact: https://docs.mcp-use.com/v2/typescript/server/deployment/mcp-use
- Connecting clients: https://docs.manufact.com/dashboard/connect-clients
- Manufact test suites: https://manufact.com/blog/mcp-evals-coding-agents
- Inkbox docs: https://inkbox.ai/docs/llms.txt
- Inkbox Voice AI: https://inkbox.ai/docs/capabilities/phone/hosted-call-agent.md
- Inkbox iMessage: https://inkbox.ai/docs/capabilities/imessage.md